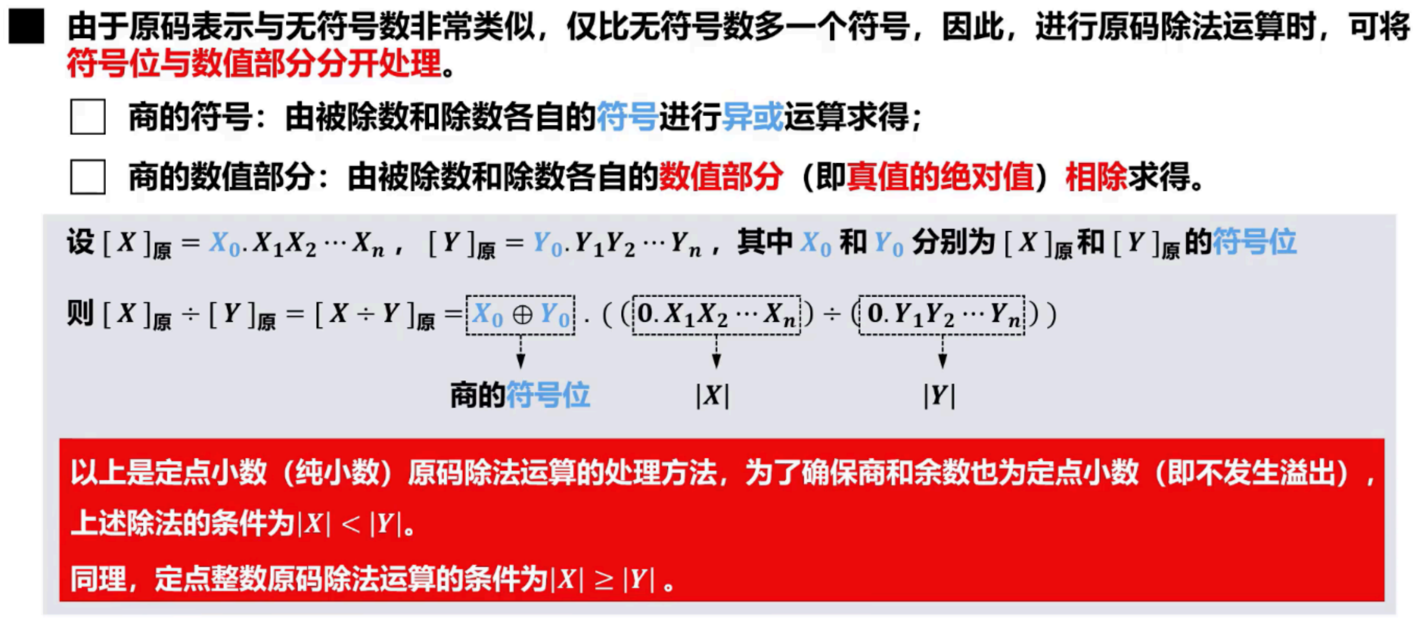
3.15 定点数的除法运算——原码除法——恢复余数法

考纲定位：计算机组成原理 · 第 3 章 运算方法（3.15）｜重要度 ★★★★ | 常见考法：选择题/大题（恢复余数、上商、左移、余数符号）

一句话核心：每次左移后试减除数；够减商 1，不够减商 0 并把除数加回去恢复余数

一、法则



■ 由于原码表示与无符号数非常类似，仅比无符号数多一个符号，因此，进行原码除法运算时，可将符号位与数值部分分开处理。

- □ 商的符号：由被除数和除数各自的符号进行异或运算求得；
- □ 商的数值部分：由被除数和除数各自的数值部分（即真值的绝对值）相除求得。

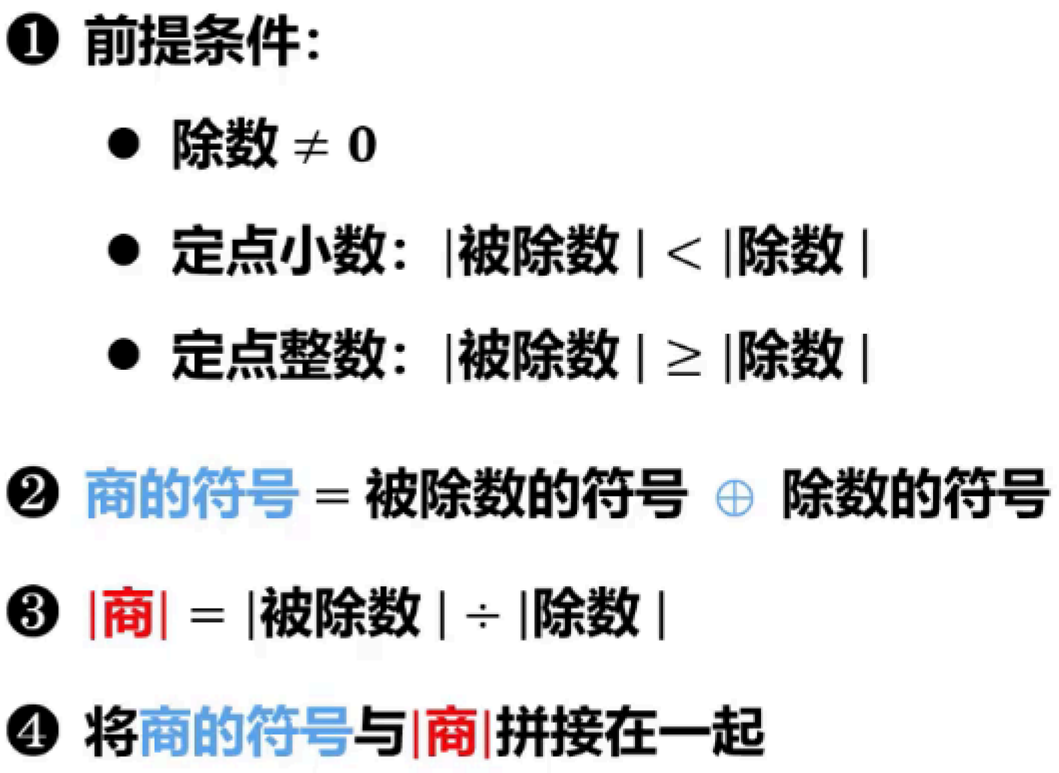
设 $[X]_{原} = X_0.X_1X_2\cdots X_n$，$[Y]_{原} = Y_0.Y_1Y_2\cdots Y_n$，其中 X_0 和 Y_0 分别为 $[X]_{原}$ 和 $[Y]_{原}$ 的符号位

则 $[X]_{原} \div [Y]_{原} = [X \div Y]_{原} = \underbrace{X_0 \oplus Y_0}_{商的符号位}.((\underbrace{0.X_1X_2\cdots X_n}_{|X|}) \div (\underbrace{0.Y_1Y_2\cdots Y_n}_{|Y|}))$

以上是定点小数（纯小数）原码除法运算的处理方法，为了确保商和余数也为定点小数（即不发生溢出），上述除法的条件为 $|X| < |Y|$。

同理，定点整数原码除法运算的条件为 $|X| \geq |Y|$。

简单而言



❶ 前提条件：

- 除数 $\neq 0$
- 定点小数：$|被除数| < |除数|$
- 定点整数：$|被除数| \geq |除数|$

❷ 商的符号 = 被除数的符号 $\oplus$ 除数的符号

❸ $|商| = |被除数| \div |除数|$

❹ 将商的符号与|商|拼接在一起

原码除法

- 商符 = 被除数符号 ⊕ 除数符号
- 数值位 = 绝对值相除
- 余数符号与被除数相同

步骤

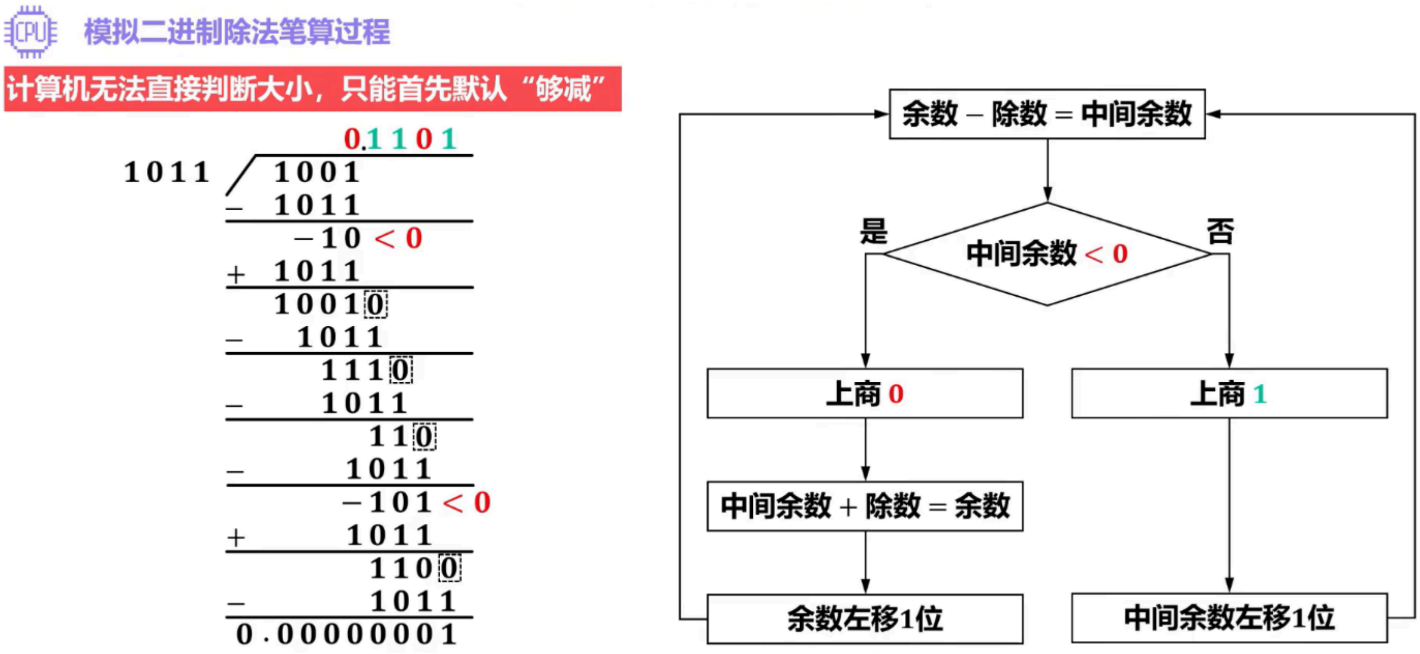
1. 求商符
2. 取绝对值，初始化余数
3. 余数左移 1 位
4. 试减除数
5. 够减：商 1，余数为差；不够减：商 0，恢复余数（加回除数）
6. 重复 n 次，得到 n 位商

易错

- 恢复余数法的“恢复”是把除数加回去，不是重新取被除数
- 商符单独异或，不参与数值除法
- 余数符号与被除数相同，不是与除数相同

二、计算机底层的除法运算逻辑

默认够减，依据结果正负，来确定上商数字以及是否恢复余数


CPU
模拟二进制除法笔算过程
计算机无法直接判断大小，只能首先默认“够减”
0.1 1 0 1
1011
1001
– 1011
–10 < 0
+ 1011
10010
– 1011
1110
– 1011
110
– 1011
–101 < 0
+ 1011
1100
– 1011
0.00000001
余数 – 除数 = 中间余数
中间余数 < 0
是
否
上商 0
上商 1
中间余数 + 除数 = 余数
余数左移1位
中间余数左移1位

判据

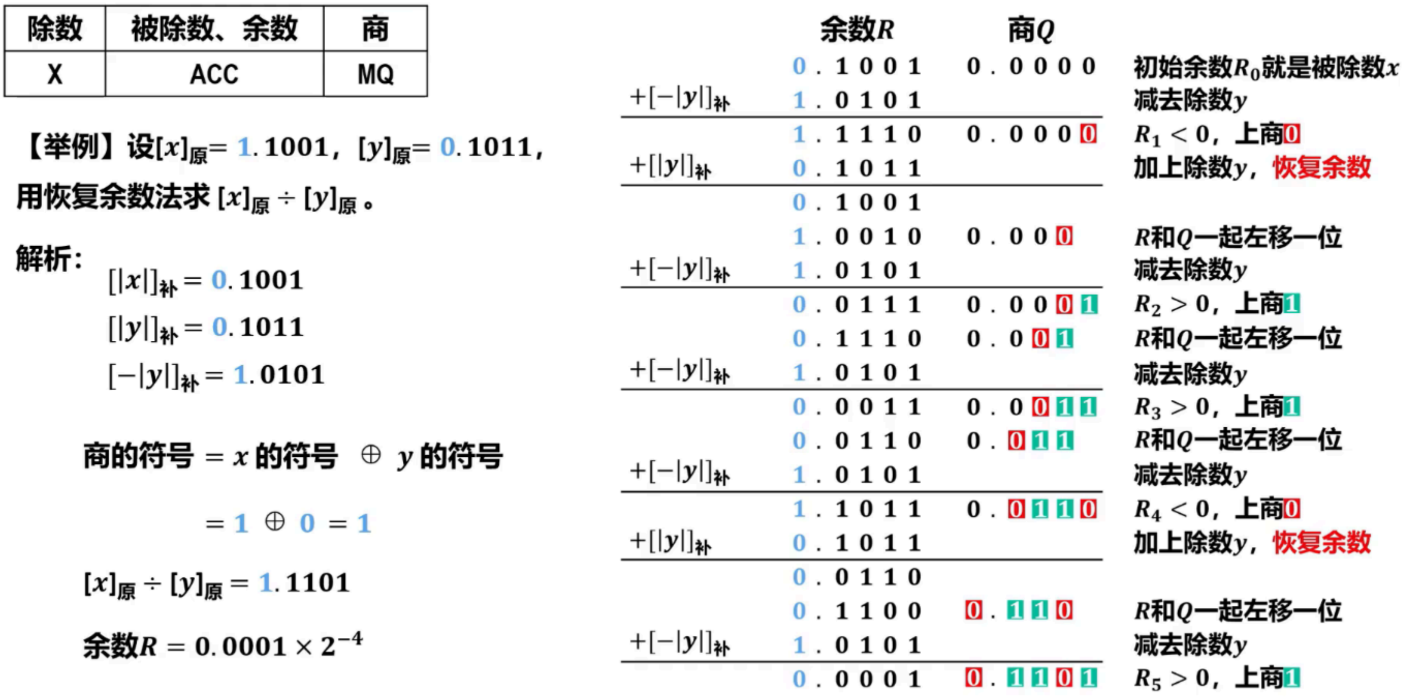
- 试减后余数 ≥ 0：够减，商 1，保留差
- 试减后余数 < 0：不够减，商 0，加回除数恢复余数

硬件

- 余数寄存器 R
- 除数寄存器 M

- 商寄存器 Q
- ALU：加减
- 计数器：控制位数

三、举例



除数	被除数、余数	商
X	ACC	MQ

【举例】设$[x]_{原}= 1.1001$，$[y]_{原}= 0.1011$，用恢复余数法求 $[x]_{原} \div [y]_{原}$。

解析：

$$[|x|]_{补} = 0.1001$$
$$[|y|]_{补} = 0.1011$$
$$[-|y|]_{补} = 1.0101$$

$$商的符号 = x\ 的符号 \oplus y\ 的符号$$
$$= 1 \oplus 0 = 1$$

$$[x]_{原} \div [y]_{原} = 1.1101$$

$$余数R = 0.0001 \times 2^{-4}$$

	余数R	商Q	
	0 . 1 0 0 1	0 . 0 0 0 0	初始余数R_0就是被除数x
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	1 . 1 1 1 0	0 . 0 0 0 0	$R_1 < 0$，上商0
$+[\|y\|]_{补}$	0 . 1 0 1 1		加上除数y，恢复余数
	0 . 1 0 0 1		
	1 . 0 0 1 0	0 . 0 0 0	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	0 . 0 1 1 1	0 . 0 0 0 1	$R_2 > 0$，上商1
	0 . 1 1 1 0	0 . 0 0 1	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	0 . 0 0 1 1	0 . 0 0 1 1	$R_3 > 0$，上商1
	0 . 0 1 1 0	0 . 0 1 1	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	1 . 1 0 1 1	0 . 0 1 1 0	$R_4 < 0$，上商0
$+[\|y\|]_{补}$	0 . 1 0 1 1		加上除数y，恢复余数
	0 . 0 1 1 0		
	0 . 1 1 0 0	0 . 1 1 0	R和Q一起左移一位
$+[-\|y\|]_{补}$	1 . 0 1 0 1		减去除数y
	0 . 0 0 0 1	0 . 1 1 0 1	$R_5 > 0$，上商1

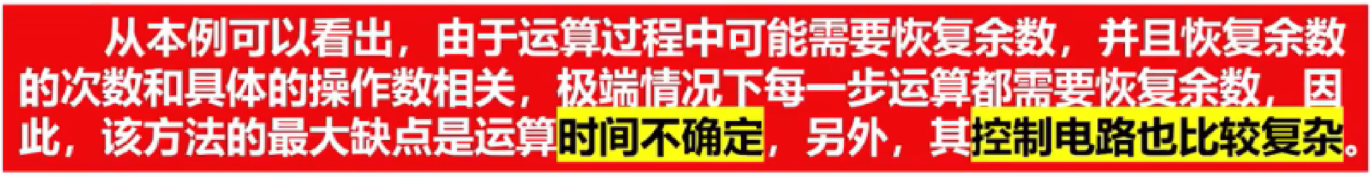
执行要点

1. 写出被除数、除数的绝对值
2. 每步左移余数，试减除数
3. 根据余数符号上商
4. 负余数要恢复
5. 最后拼接商符与商数值

易错

- 左移后试减，不是先减再左移
- 余数为负时必须恢复，否则下一步错误
- 商的位数等于数值位位数，不含符号位

四、恢复余数法的缺点



从本例可以看出，由于运算过程中可能需要恢复余数，并且恢复余数的次数和具体的操作数相关，极端情况下每一步运算都需要恢复余数，因此，该方法的最大缺点是运算时间不确定，另外，其控制电路也比较复杂。

- 每次不够减都要加回除数，多一次加法

- 步骤不固定，控制复杂
- 速度较慢
- 改进方法：不恢复余数法（加减交替法）

知识关联：不恢复余数法（3.16 定点数的除法运算——原码的除法——不恢复余数法（加减交替法））；补码除法（3.17 定点数的除法运算——补码的除法——不恢复余数法）；补码加减（3.4 定点数的加减运算——补码运算规则）